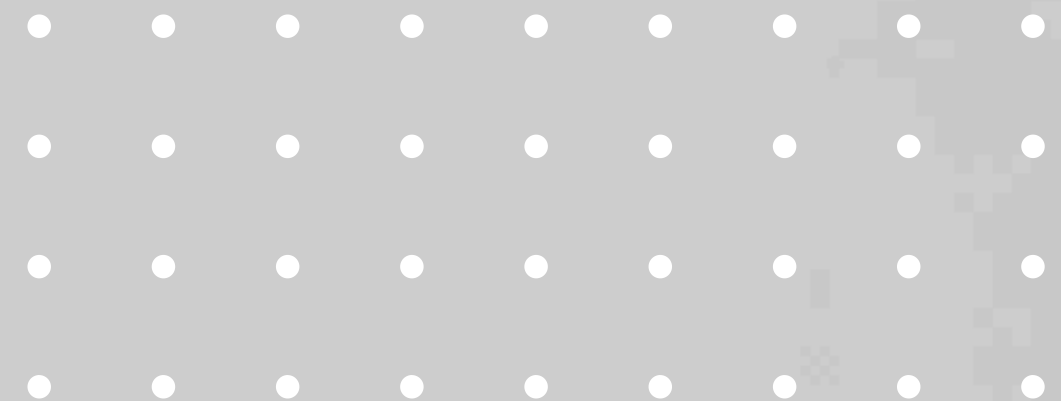


VISUALIZACION Y *EXPORTACIONES* de reconstrucciones COLMAP

SEBASTIAN MUÑOZ → JUMUNOZLE@UNAL.EDU.CO
CARLOS CAMACHO → CACAMACHO@UNAL.EDU.CO
JUAN DANIEL RAMÍREZ → JUARAMRIEZMO@UNAL.EDU.CO
SERGIO DAVID LOPEZ → SLOPEZPA@UNAL.EDU.CO



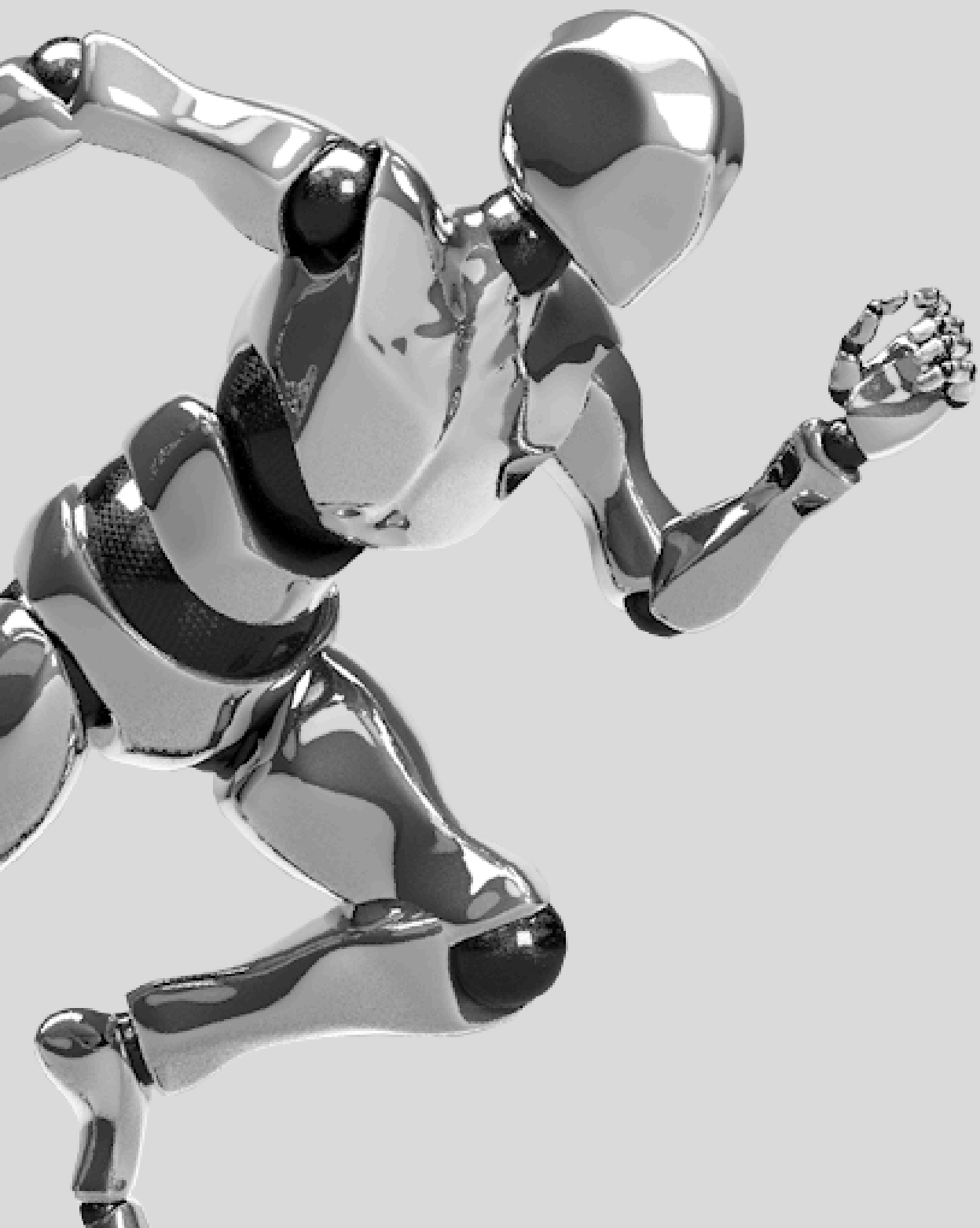
ÍNDICE

INTRO A COLMAP

EXPO DE DATOS DESDE COLMAP

INTEGRACIÓN CON THREE.JS

INTEGRACION EN APPS 3D



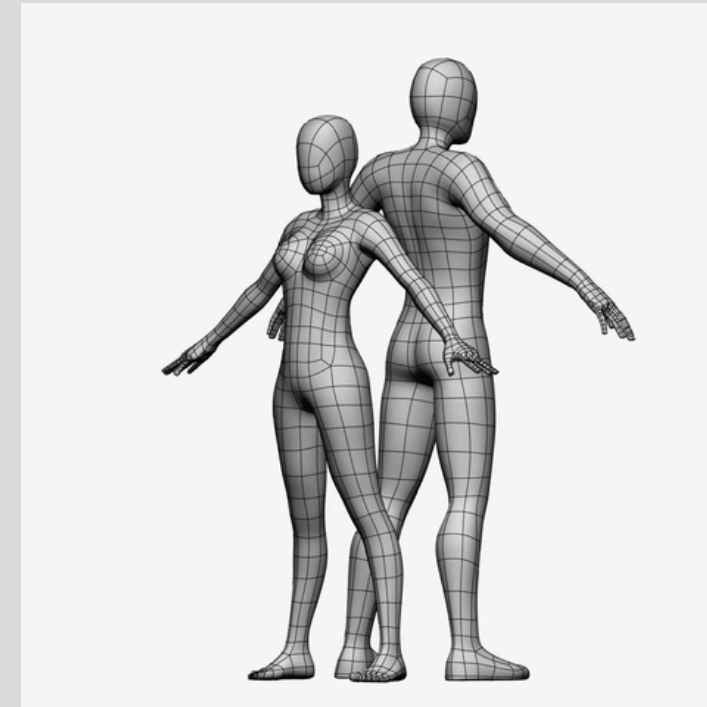
ARCHIVOS EXPORTABLE S DESDE COLMAP

- COLMAP genera reconstrucciones 3D a partir de imágenes.
- Exporta nubes de puntos y mallas en varios formatos:

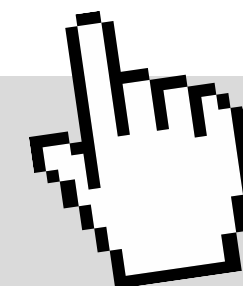
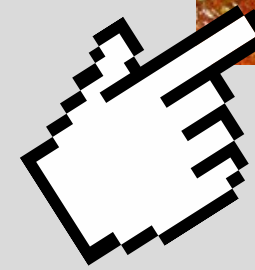
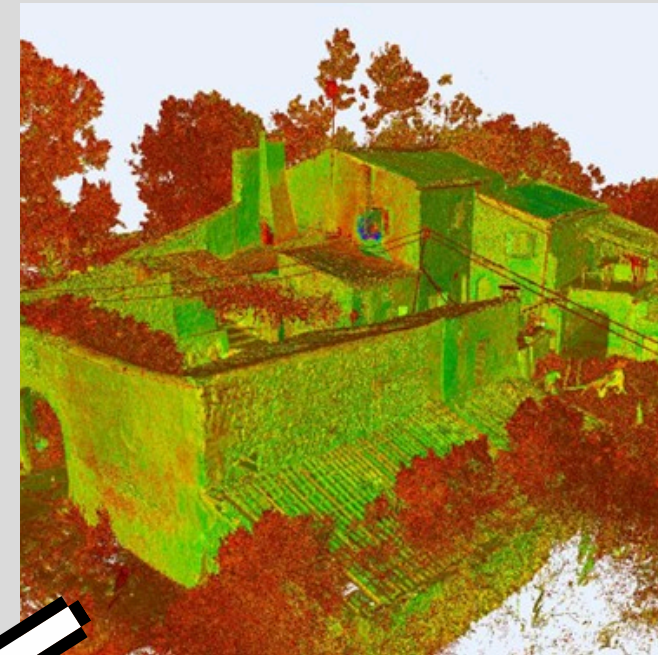
.PLY: NUBE DE PUNTOS O MALLA DENSA.

.OBJ: MALLA CON TEXTURA.

.FBX: USADO EN ENTORNOS COMO UNITY.



QUÉ
RESULTADOS
GENERA
COLMAP?

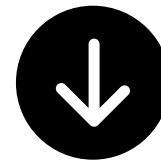


PARÁMETROS DE RECONSTRUCCIÓN EN COLMAP



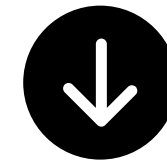
Match features:

- Extracción: SIFT (por defecto).
- Emparejamiento: exhaustive, sequential, spatial, etc.



Structure-from-Motion:

- Estimación de cámaras: Pinhole, Simple Radial, etc.
- Bundle Adjustment para optimizar cámaras y puntos 3D.
- Multi



Multi-view Stereo (MVS):

- Genera mapas de profundidad.
- Fusión produce fused.ply (nube densa).

1. Completa la reconstrucción.
2. En el GUI: File > Export model
 - Elegir: Formato: .ply, .obj, .fbx
 - Tipo: sparse, dense, mesh

```
colmap model_converter \  
  --input_path ./sparse/0 \  
  --output_path ./output \  
  --output_type PLY
```

¿CÓMO EXPORTAR?

1. A veces se requiere convertir el archivo .ply a .obj o .fbx:

- ```
import open3d as o3d
pcd = o3d.io.read_point_cloud("modelo.ply")
o3d.io.write_point_cloud("modelo.obj", pcd)
```



# EJEMPLOS *PRÁCTICOS* *CON UNITY*

The screenshot shows the Unity 2019.3.1f1 interface. The Hierarchy panel on the left shows the SampleScene with a Main Camera and a Directional Light. The Scene view in the center shows a 2D environment with a sun and a camera. The Package Manager on the right lists various packages, with COLIBRI VR 1.0.0 highlighted. The Package Details panel on the far right shows the COLIBRI VR package information, including its version, name, links, author, and samples. A red arrow points to the package name in the Package Details panel.

**Inspector** **Package Manager** **Project Settings**

**Package Manager**

| Package Name           | Version      |
|------------------------|--------------|
| 2D Animation           | 3.1.1        |
| 2D Common              | 2.0.2        |
| 2D Path                | 2.0.4        |
| 2D Pixel Perfect       | 2.0.4        |
| 2D PSD Importer        | 2.1.3        |
| 2D Sprite              | 1.0.0        |
| 2D SpriteShape         | 3.0.10       |
| 2D Tilemap Editor      | 1.0.0        |
| Adaptive Performance   | 1.0.1        |
| Addressables           | 1.1.10       |
| Ads                    | 2.0.8        |
| Alembic                | 1.0.6        |
| Analytics Library      | 3.3.5        |
| Android Logcat         | 1.0.0        |
| AP Samsung Android     | 1.0.1        |
| AR Foundation          | 2.1.8        |
| AR Subsystems          | 2.1.2        |
| ARCore XR Plugin       | 2.1.8        |
| ARKit Face Tracking    | 1.0.7        |
| ARKit XR Plugin        | 2.1.9        |
| Asset Bundle Browser   | 1.7.0        |
| Burst                  | 1.2.3        |
| Cinemachine            | 2.5.0        |
| <b>COLIBRI VR</b>      | <b>1.0.0</b> |
| Core RP Library        | 7.1.8        |
| Custom NUnit           | 1.0.0        |
| Google Resonance Audio | 2.0.0        |
| Google VR Android      | 2.0.0        |
| Google VR iOS          | 2.0.1        |

**COLIBRI VR**

Version 1.0.0

**Name**  
com.mines-paristech.colibri-vr

**Links**  
[View documentation](#)  
[View changelog](#)  
[View licenses](#)

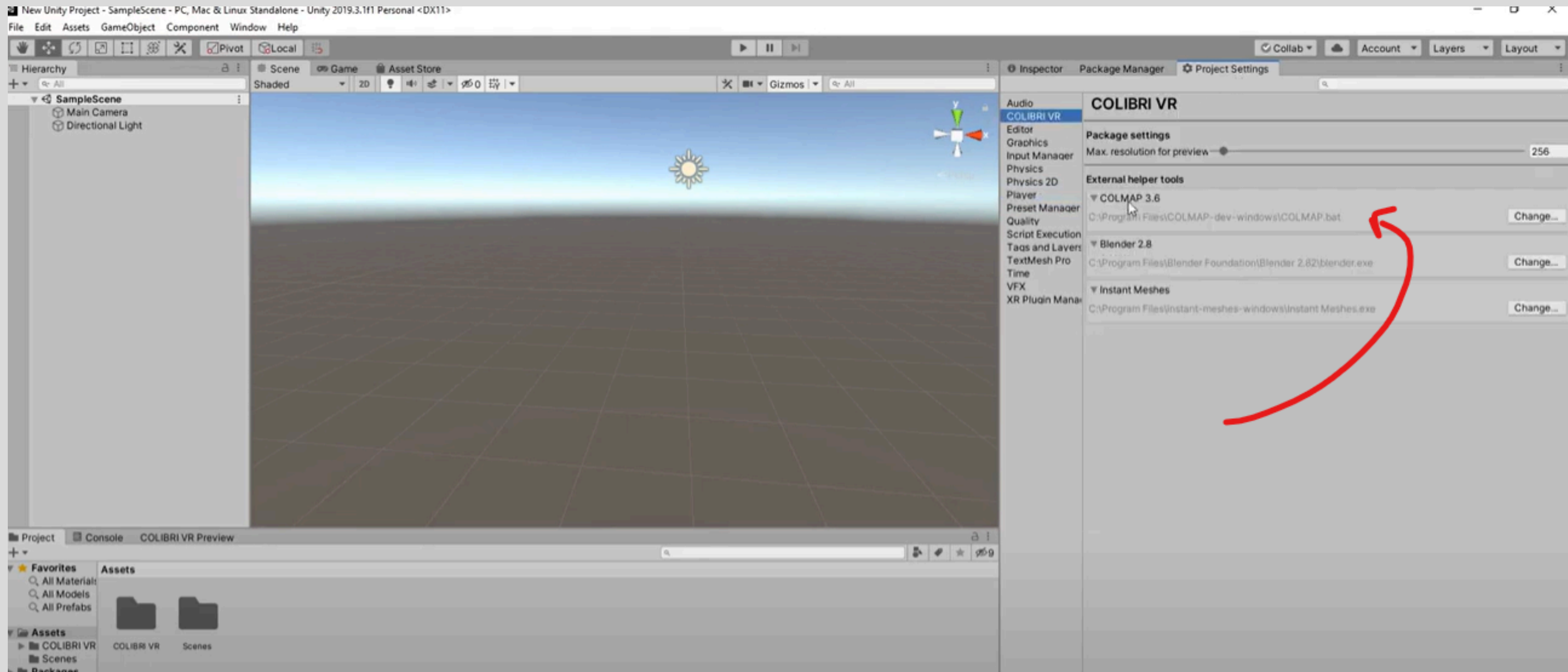
**Author**  
Grégoire Dupont de Dinechin

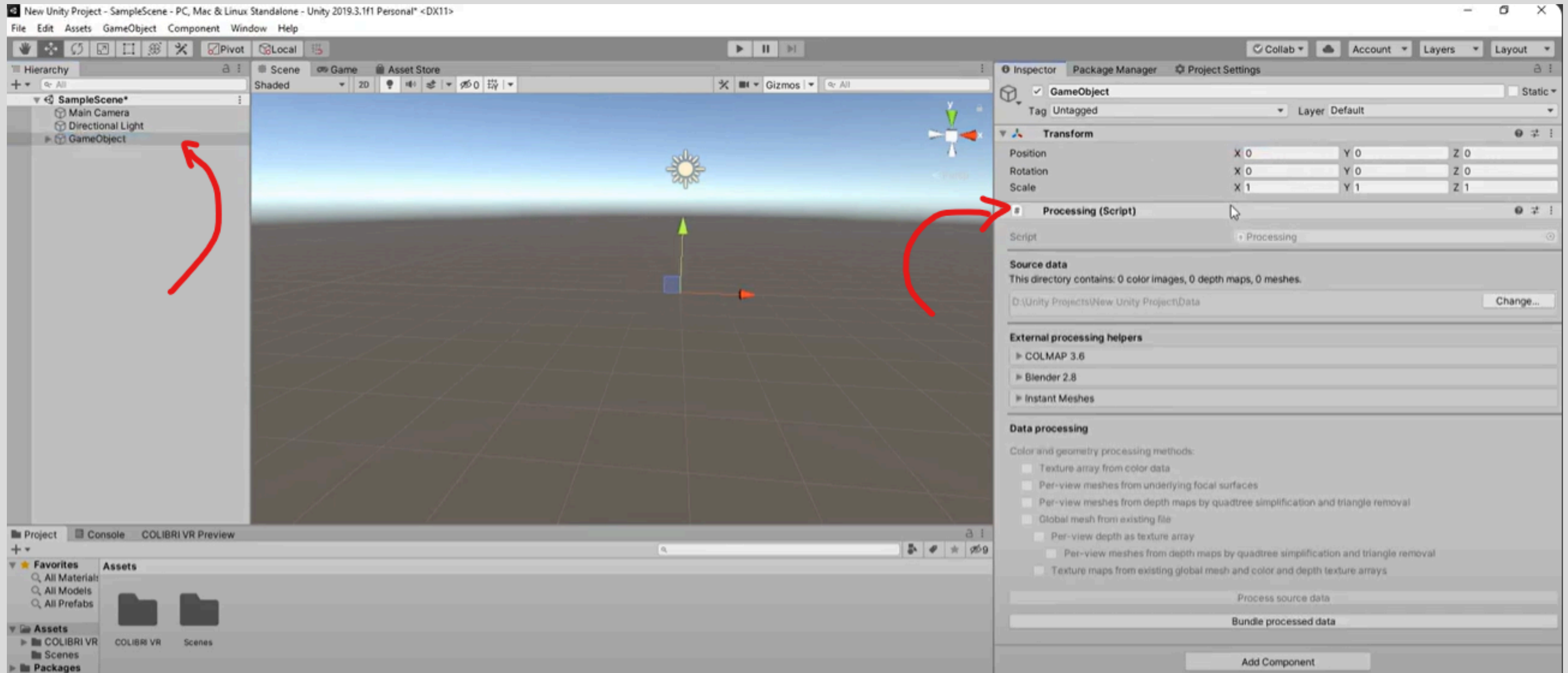
Unity package for COLIBRI VR, the Core Open Lab on Image-Based Rendering Innovation for Virtual Reality.

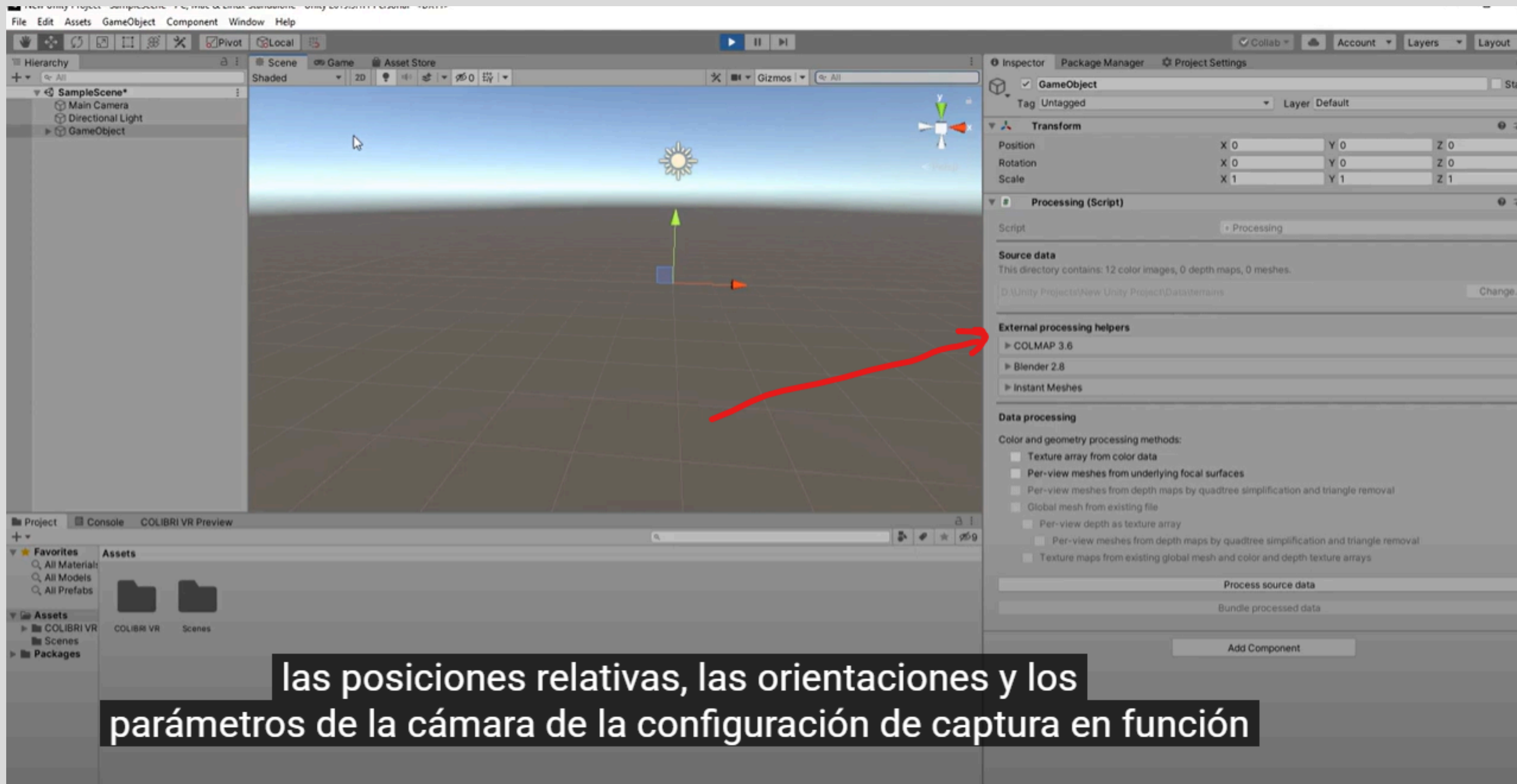
**Samples**

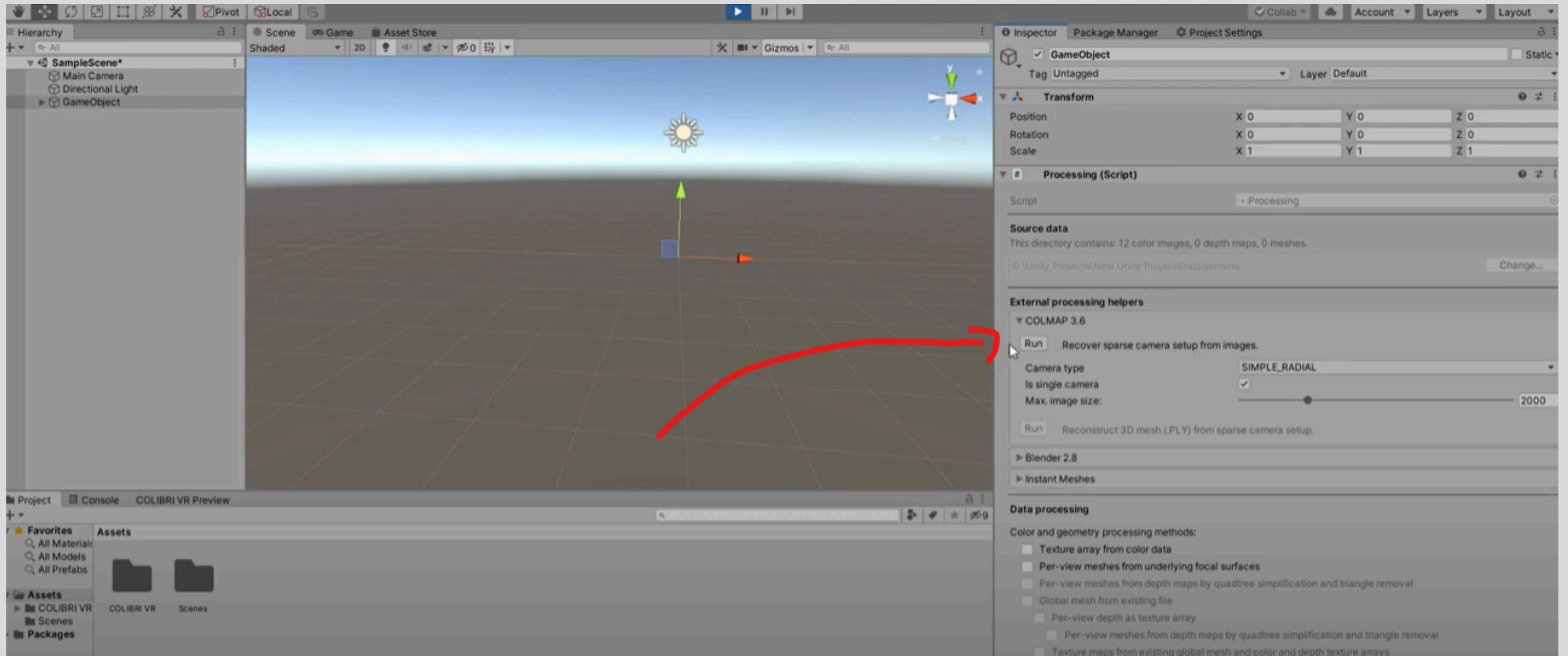
| Sample Name | Size      | Action                              |
|-------------|-----------|-------------------------------------|
| Demo Scenes | 860,56 KB | <a href="#">Import into Project</a> |



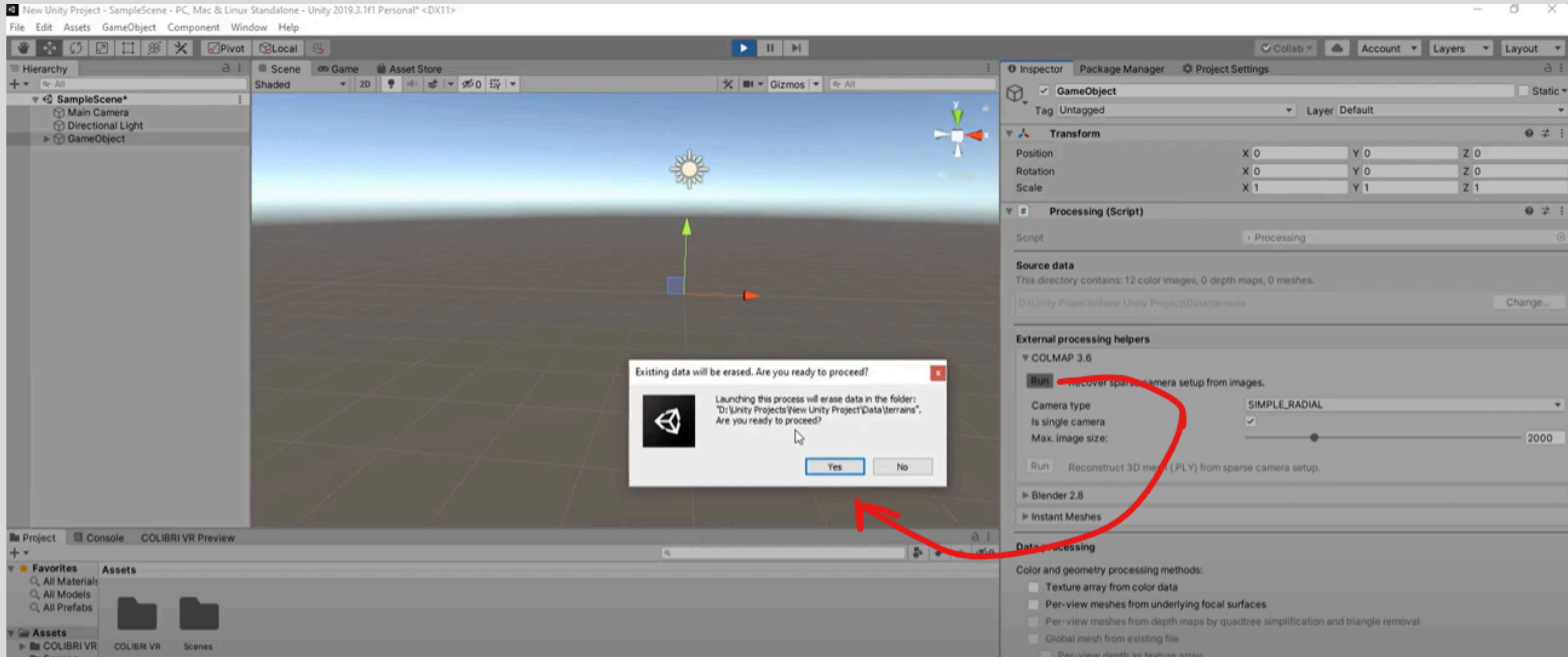


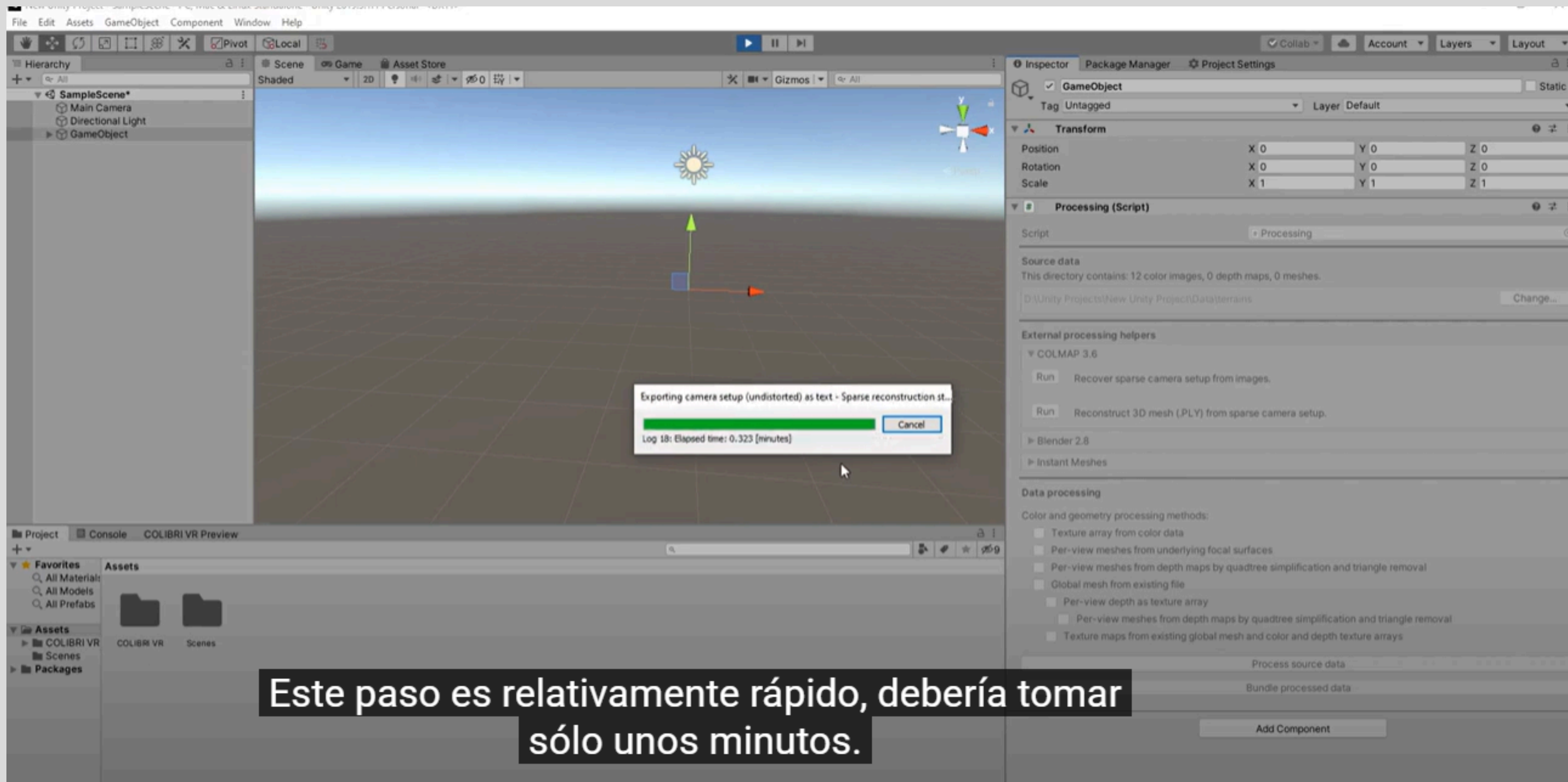


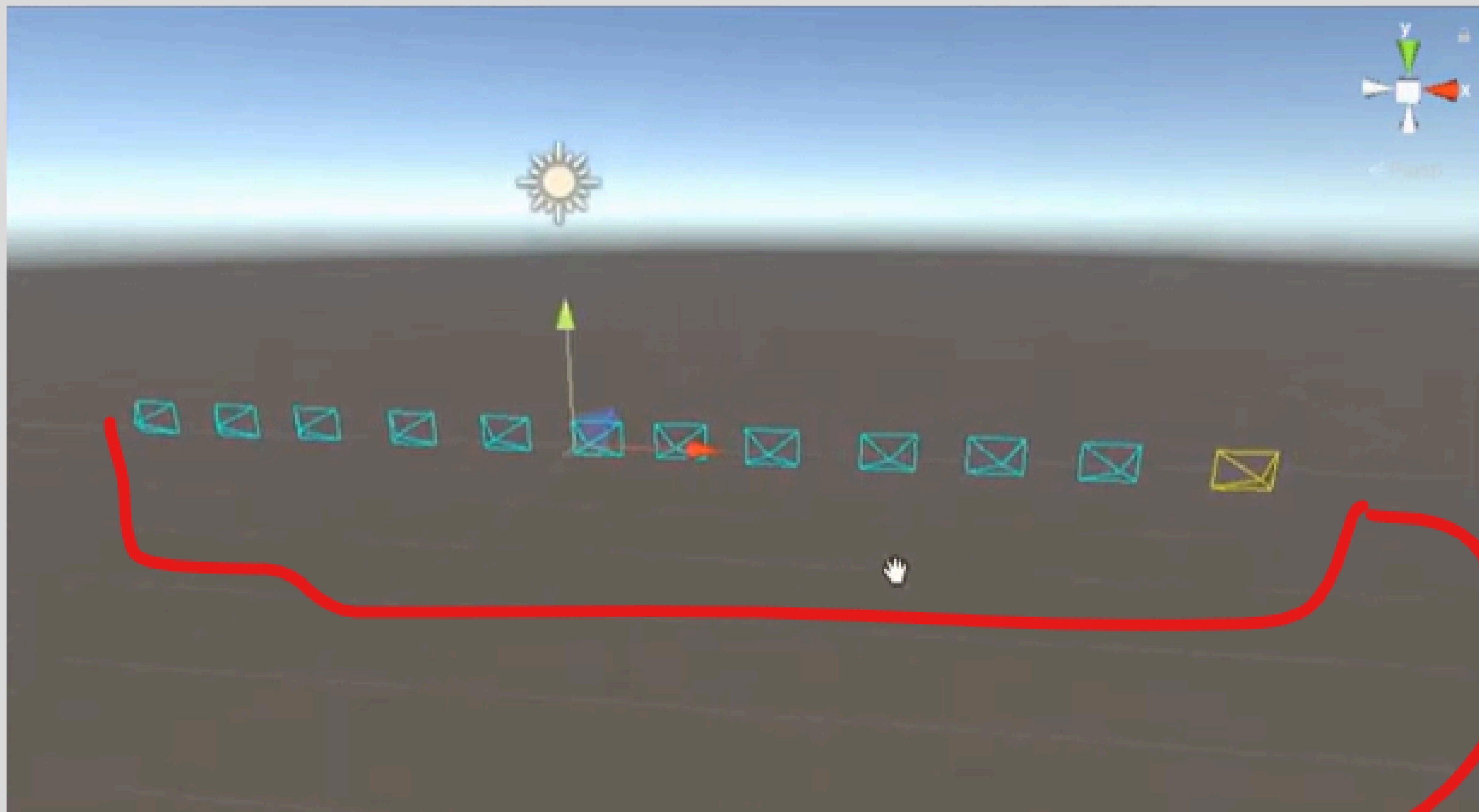










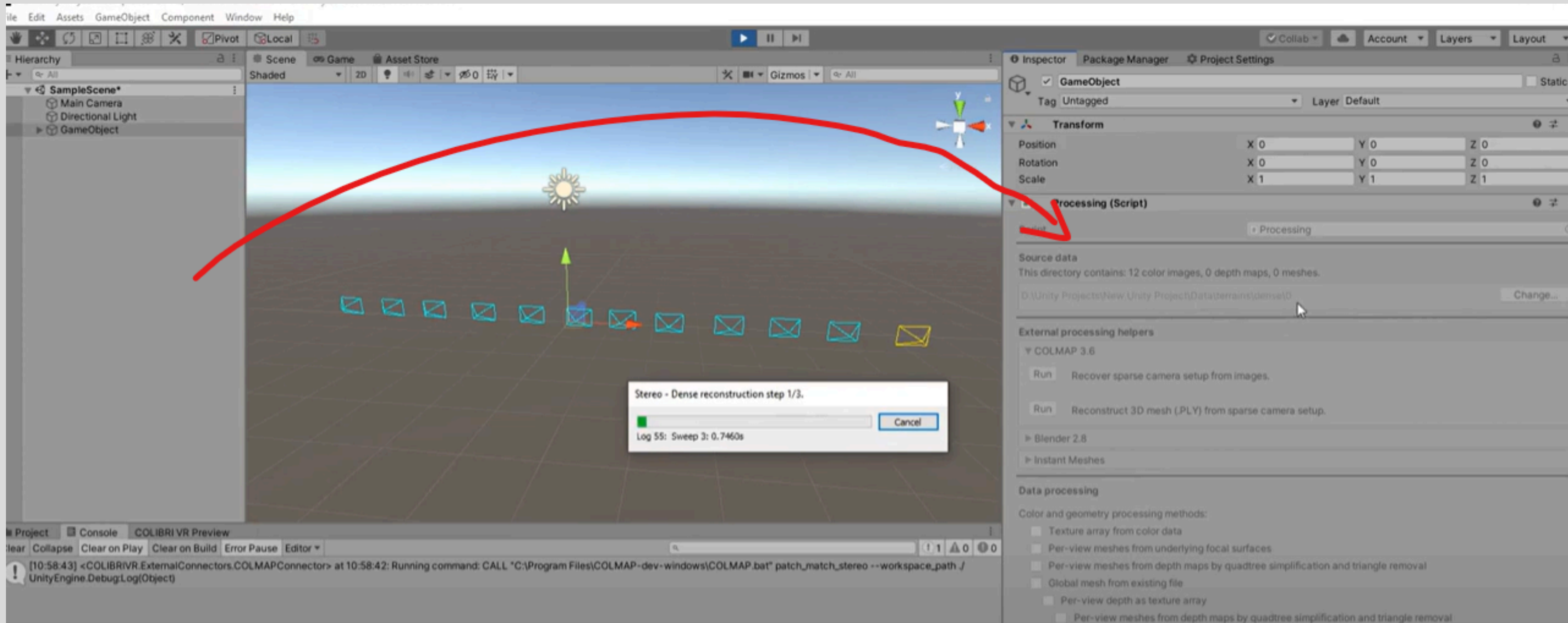


The screenshot shows the Unity 2019.3.1f1 Personal interface. The Hierarchy panel on the left shows a 'SampleScene\*' with 'Main Camera', 'Directional Light', and 'GameObject'. The Scene view in the center shows a grid with a sun and a row of camera frustums. A red circle highlights this row, and a red arrow points from it to the 'Reconstruct 3D mesh (PLY) from sparse camera setup' button in the Inspector panel. The Inspector panel on the right shows the 'GameObject' component with 'Tag: Untagged' and 'Layer: Default'. Below the Transform component is the 'Processing (Script)' component, which has a 'Script' dropdown set to 'Processing'. Under 'Source data', it says 'This directory contains: 12 color images, 0 depth maps, 0 meshes.' and shows the path 'D:\Unity Projects\New Unity Project\Data\terrains\dense\0'. Under 'External processing helpers', there are three options: 'COLMAP 3.6' (with a 'Run' button and description 'Recover sparse camera setup from images.'), 'Blender 2.8', and 'Instant Meshes'. The 'Reconstruct 3D mesh (PLY) from sparse camera setup.' button is highlighted with a red box. Under 'Data processing', there are several checkboxes for 'Color and geometry processing methods'.

Inspector Panel:

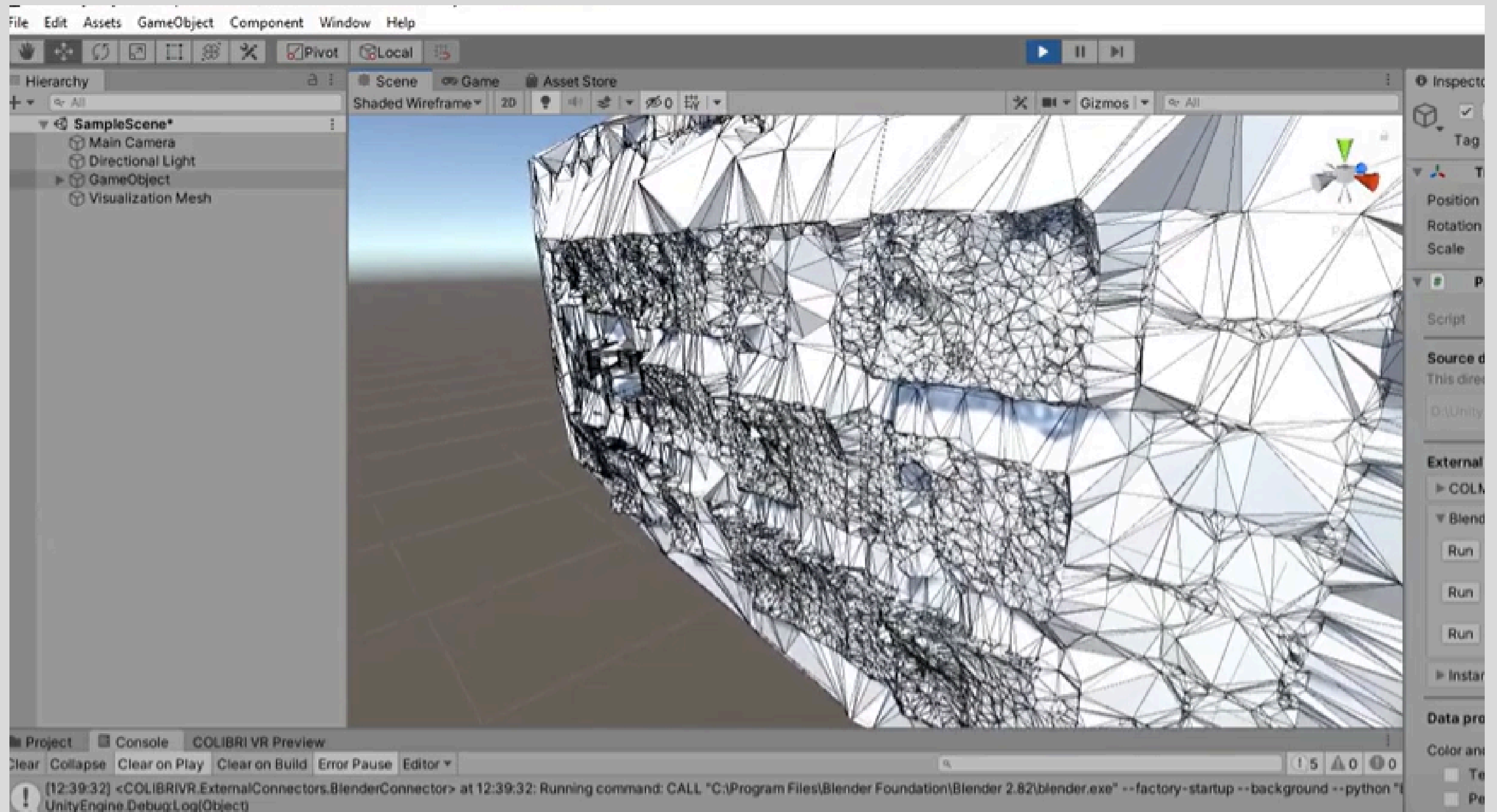
- GameObject
- Tag: Untagged
- Layer: Default
- Transform
  - Position: X 0, Y 0, Z 0
  - Rotation: X 0, Y 0, Z 0
  - Scale: X 1, Y 1, Z 1
- Processing (Script)
  - Script: Processing
  - Source data
    - This directory contains: 12 color images, 0 depth maps, 0 meshes.
    - D:\Unity Projects\New Unity Project\Data\terrains\dense\0
  - External processing helpers
    - COLMAP 3.6
      - Run: Recover sparse camera setup from images.
      - Run: Reconstruct 3D mesh (PLY) from sparse camera setup.
    - Blender 2.8
    - Instant Meshes
  - Data processing
    - Color and geometry processing methods:
      - ☐ Texture array from color data
      - ☐ Per-view meshes from underlying focal surfaces
      - ☐ Per-view meshes from depth maps by quadtree simplification and triangle removal
      - ☐ Global mesh from existing file
      - ☐ Per-view depth as texture array
      - ☐ Per-view meshes from depth maps by quadtree simplification and triangle removal







Bien, ahora tenemos un archivo "meshed-delaunay.obj" que podemos usar en Unity.



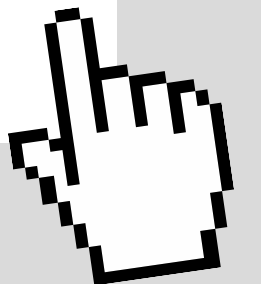


UNA VEZ QUE COLMAP HA GENERADO UNA RECONSTRUCCIÓN 3D, YA SEA UNA NUBE DE PUNTOS O UNA MALLA DENSA, EL SIGUIENTE PASO ES PREPARAR ESOS DATOS PARA SU USO EN ENTORNOS DE VISUALIZACIÓN COMO UNITY, THREE.JS O BLENDER. PARA ELLO, ES NECESARIO EXPORTAR, AJUSTAR Y LIMPIAR ADECUADAMENTE LOS MODELOS.

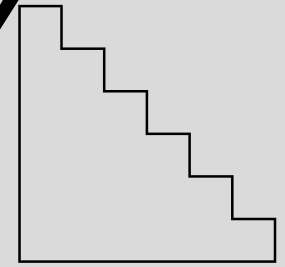
## Exportación desde COLMAP 🔍

COLMAP permite exportar los resultados en varios formatos estándar o mediante herramientas externas en .fbx. Estos formatos son ampliamente compatibles con aplicaciones 3D modernas. Al exportar, es común utilizar:

- .ply para nubes de puntos crudas o filtradas.
- .obj para mallas texturizadas generadas con Delaunay o Poisson.
- .fbx si se requiere compatibilidad directa con motores como Unity.



# AJUSTE DE ESCALA, ORIENTACIÓN Y LIMPIEZA (THREE.JS)



1. Cargar la nube de puntos o malla



```
import { GLTFLoader } from 'three/examples/jsm/loaders/GLTFLoader';
import { PointLight, Scene, PerspectiveCamera, WebGLRenderer } from 'three';

const loader = new GLTFLoader();
const scene = new Scene();

loader.load('/models/colmap_output.glb', (gltf) => {
 const model = gltf.scene;

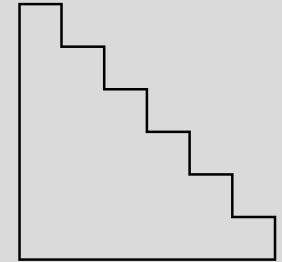
 // Ajustes clave:
 model.scale.set(0.01, 0.01, 0.01); // ← Ajusta la escala
 model.rotation.set(-Math.PI / 2, 0, 0); // ← Orientación (si Z apunta arriba)
 model.position.set(0, 0, 0); // ← Posición inicial

 scene.add(model);
});
```

Cuando se exporta desde COLMAP (.ply, .obj), se puede obtener ruido, duplicados o geometría sobrante. Para integrar esto correctamente en una app 3D, es útil filtrar o limpiar antes o durante la carga.

¿Qué es "limpieza"?

1. Eliminar puntos o geometría fuera de un rango espacial (por ejemplo, demasiado alejados).
2. Filtrar por bounding box: sólo conservar una región 3D.
3. Ocultar mallas específicas si son subcomponentes innecesarios.



Opcion 1: Limpieza con bounding box en Three.js (si ya está cargado)

```
model.traverse((child) => {
 if (child.isMesh) {
 child.geometry.computeBoundingBox();
 const box = child.geometry.boundingBox;

 const min = box.min;
 const max = box.max;

 // Si el objeto está fuera de un rango aceptable, lo ocultamos
 const isOutside = max.z > 5 || min.z < -2 || max.x > 5 || min.x < -5;
 if (isOutside) {
 child.visible = false;
 }
 }
});
```

Esto permite en tiempo real desactivar parte de la geometría cargada

Opción 2: Limpieza previa con herramientas gráficas

En CloudCompare:

Cargar .ply → Usar "Crop Tool" para seleccionar una región 3D.

Aplicar filtro por densidad o distancia.

Exportar la versión reducida.

En Blender:

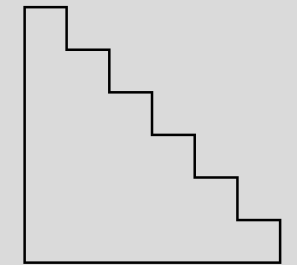
Importar .ply o .obj.

Usar modo Edit → Box select para borrar puntos aislados.

Exportar como .glb o .gltf limpio.



Opción 3: Limpieza automatizada por densidad (en Python)  
Si se trabaja por fuera de Three.js, se puede hacer la limpieza con Open3D:



```
import open3d as o3d

pcd = o3d.io.read_point_cloud("dense.ply")
cl, ind = pcd.remove_statistical_outlier(nb_neighbors=20, std_ratio=2.0)

pcd_clean = pcd.select_by_index(ind)
o3d.io.write_point_cloud("cleaned_dense.ply", pcd_clean)
```

Esto elimina puntos dispersos basados en la densidad local.



***GRACIAS POR LA  
ATENCIÓN***

